A Numerical-Optimization View of Context Selection for Document-Grounded Generation
Technical Report for `wordcopilot-context-lab`

March 14, 2026

Abstract

This report presents a mathematically explicit analysis of the repository `wordcopilot-context-lab`. The repository implements two document-grounded generation systems over QMSum: a *baseline* local-context system and a *context-aware* system that augments local evidence with a global document summary and a greedy maximal marginal relevance (MMR) selector under a word budget. The goal of this report is not merely to describe the code, but to formulate the computational problem : first the inverse problem is stated, then the modeling assumptions are made explicit, then the implemented algorithm is derived, and finally the numerical behavior is analyzed on the observed experiment.

1 Introduction

Document-grounded generation may be formulated as a constrained approximation problem. A user provides a query q and a long document D. The system must generate an answer y that is simultaneously relevant to q, faithful to information contained in D, and computationally efficient. In modern large-language-model systems these desiderata are mutually coupled: higher answer quality often requires larger context, but larger context incurs latency, redundancy, and loss of factual sharpness.

The repository `wordcopilot-context-lab` studies exactly this tension. It compares two systems:

(i) a *baseline* system that retrieves a set of candidate chunks and uses only the highest-scoring chunk for generation;

(ii) a *context-aware* system that supplements the answer prompt with a global summary of the document and selects several chunks by a greedy MMR procedure subject to a word budget.

This is a small repository, but it is conceptually rich because it contains the essential ingredients of contemporary retrieval-augmented generation: chunking, dense retrieval, diversity-aware context packing, prompting, generation, and evaluation. The natural mathematical question is the following:

> *When does increasing context improve the approximation of the latent gold answer, and when does it merely increase computational burden while degrading grounding?*

The present numerical evidence shows that this question cannot be answered by intuition alone. On the measured sample, the context-aware system improves one surface-form metric but degrades grounding, terminology coverage, and latency. That finding is important because it demonstrates that better context selection is not automatically equivalent to better generation.

2 Problem formulation

We begin from first principles.

Definition 2.1 (Document-grounded answer generation)**.** *Let $\mathcal{D}$ denote a space of documents, $\mathcal{Q}$ a space of user queries, and $\mathcal{Y}$ a space of textual answers. A document-grounded generation system is a map*

$$\mathcal{A} : \mathcal{D} \times \mathcal{Q} \to \mathcal{Y}, \qquad (D, q) \mapsto \hat{y},$$

where the produced answer $\hat{y}$ should approximate a latent reference answer $y^\star(D, q)$ using only information contained in D.

The adjective *grounded* means that the approximation error is not measured purely at the linguistic level. The system must also respect an evidence constraint: information in $\hat{y}$ should be attributable to the document.

Definition 2.2 (Latent constrained objective)**.** *For a fixed pair (D, q), define the ideal utility functional*

$$\mathcal{J}(y; D, q) = \alpha \,\mathrm{Rel}(y, y^\star) + \beta \,\mathrm{Ground}(y; D) + \gamma \,\mathrm{Term}(y; D) - \delta \,\mathrm{Cost}(y; D, q),$$

where

- Rel *measures relevance to the reference answer,*
- Ground *measures evidential support inside D,*
- Term *measures recovery of important document terminology,*
- Cost *measures computational burden, in practice latency and context size.*

The latent optimal answer is

$$y^{\mathrm{opt}} = \arg\max_{y \in \mathcal{Y}} \mathcal{J}(y; D, q).$$

This functional is not optimized directly in the repository, but it is the correct abstract problem. The implemented pipeline approximates this optimization through several intermediate steps.

2.1 Historical simplification

Early extractive or short-context systems implicitly solved a surrogate problem of the form

$$\hat{y} = G_\theta(q, c),$$

where c is a single local context segment selected from the document. This formulation is computationally simple but structurally fragile: if the information required to answer q is distributed across multiple non-adjacent passages, then no single local segment can represent the full evidence set.

This motivates a two-level decomposition:

$$D \longrightarrow \text{candidate contexts} \longrightarrow \text{packed context} \longrightarrow \hat{y}.$$

The repository follows exactly this principle.

3 From full documents to optimization variables

3.1 Document chunking

The document D is first converted into a sequence of word tokens

$$D = (w_1, w_2, \dots, w_N).$$

Given two hyperparameters

$$L = \texttt{chunk_size_words}, \qquad s = \texttt{stride_words},$$

the implementation defines the step size

$$\Delta = L - s.$$

In the repository configuration,

$$L = 140, \qquad s = 40, \qquad \Delta = 100.$$

Hence each chunk contains at most 140 words, and neighboring chunks overlap by 40 words.

Definition 3.1 (Chunk family)**.** *Let $k \in \{0, 1, 2, \dots\}$ and define*

$$I_k = \{k\Delta + 1,\ k\Delta + 2,\ \dots,\ \min(N, k\Delta + L)\}.$$

The k-th chunk is

$$C_k = (w_i)_{i \in I_k}.$$

The chunk family is the ordered set

$$\mathcal{C}(D) = \{C_0, C_1, \dots, C_{m-1}\},$$

where m is the smallest integer such that the last chunk covers w_N.

Proposition 3.2 (Coverage of the chunking rule)**.** *Every token position $j \in \{1, \dots, N\}$ belongs to at least one chunk.*

Proof. Set $k = \left\lfloor \frac{j-1}{\Delta} \right\rfloor$. Then $k\Delta + 1 \le j$. By construction,

$$j \le k\Delta + \Delta \le k\Delta + L,$$

because $\Delta = L - s \le L$. Therefore $j \in I_k$, hence w_j is contained in C_k. □

Remark 3.3. The overlap is a discrete regularization device. It reduces boundary artifacts that would otherwise arise when semantically related information lies near the edge of a chunk.

3.2 Top-term extraction and global summary

The context-aware system also computes a global summary. Let $\mathcal{T}(D)$ be the set of candidate tokens obtained by lowercasing, regular-expression tokenization, and stopword removal. Let $f(t; D)$ denote the empirical count of term t. The implementation defines the top-term set

$$\mathrm{TopTerms}_K(D) = \{t_1, \dots, t_K\},$$

where the terms are the K most frequent surviving tokens. In the global-summary heuristic the code uses $K = 40$.

The document is split into sentences $S_1, \ldots, S_r$. For each sentence, let $W(S_j)$ be its set of tokenized content words. The repository scores each sentence by

$$\sigma(S_j; D) = \frac{|W(S_j) \cap \mathrm{TopTerms}_{40}(D)|}{\log(\max\{2, |S_j|\})},$$

where $|S_j|$ denotes the sentence length in words.

The global summary is obtained by:

(a) selecting the M highest-scoring sentences, with $M = 8$ by configuration;

(b) restoring them to original document order;

(c) concatenating them into a summary string.

4 Dense retrieval as a continuous relaxation

Let $\phi : \text{text} \to \mathbb{R}^d$ denote the sentence-transformer encoder used by the repository,

$$\phi = \texttt{sentence-transformers/all-MiniLM-L6-v2}.$$

The encoder returns normalized embeddings. Therefore, for any two texts u, v,

$$\|\phi(u)\|_2 = \|\phi(v)\|_2 = 1,$$

and cosine similarity reduces to a dot product.

For a query q and a chunk C_i, define the retrieval score

$$r_i = \langle \phi(q), \phi(C_i) \rangle.$$

The retriever returns the top-K_0 chunks by score, where by configuration

$$K_0 = \texttt{initial_top_k} = 6.$$

Hence the candidate set is

$$\mathcal{R}_{K_0}(q, D) = \mathrm{TopK}_i(r_i; K_0).$$

Definition 4.1 (Continuous relevance surrogate)**.** *The map $C_i \mapsto r_i$ is a continuous surrogate for symbolic relevance. It embeds a discrete linguistic retrieval problem into a Euclidean similarity geometry.*

Remark 4.2. Because the embeddings are normalized, the retrieval stage implicitly solves a nearest-neighbor problem on the unit sphere. This is important because it makes the score scale-stable and interpretable as angular alignment.

5 Context packing as a constrained optimization problem

The baseline system takes only the highest-scoring retrieved chunk. The context-aware system instead solves a diversity-aware selection problem under a word budget.

Let the retrieved candidates be

$$\mathcal{R}_{K_0}(q, D) = \{R_1, \ldots, R_{K_0}\},$$

where each R_i contains a chunk C_i, its score, and its embedding $e_i = \phi(C_i)$. Let $e_q = \phi(q)$. Let ℓ_i be the word count of chunk C_i.

The implemented selector is a greedy MMR scheme with

$$\lambda = \texttt{mmr_lambda} = 0.75,$$

final cardinality cap

$$K_1 = \texttt{final_top_k} = 3,$$

and budget

$$B = \texttt{token_budget_words} = 500.$$

5.1 Ideal combinatorial formulation

Introduce binary variables $x_i \in \{0, 1\}$ indicating whether candidate R_i is selected. A natural idealized optimization problem is

$$\max_{x \in \{0,1\}^{K_0}} \quad \lambda \sum_{i=1}^{K_0} r_i x_i - (1 - \lambda) \sum_{1 \le i < j \le K_0} \rho_{ij} x_i x_j \tag{1}$$

$$\text{subject to} \quad \sum_{i=1}^{K_0} \ell_i x_i \le B, \tag{2}$$

$$\sum_{i=1}^{K_0} x_i \le K_1, \tag{3}$$

where

$$\rho_{ij} = \langle e_i, e_j \rangle$$

measures redundancy.

The first term rewards relevance to the query; the second discourages selecting multiple near-duplicate chunks.

Remark 5.1. This problem is discrete and nonconvex because of the bilinear redundancy term $x_i x_j$. The repository does not solve it globally; instead it uses a greedy approximation.

5.2 Greedy MMR rule actually implemented

Suppose that after t greedy steps the selected set is S_t. For each unselected candidate R_i, define

$$\mathrm{red}_i(S_t) = \begin{cases} 0, & S_t = \varnothing, \\ \max\limits_{R_j \in S_t} \langle e_i, e_j \rangle, & S_t \neq \varnothing. \end{cases}$$

Then the candidate score is

$$\mathrm{MMR}_i(S_t) = \lambda \langle e_i, e_q \rangle - (1 - \lambda)\ \mathrm{red}_i(S_t).$$

At each step the algorithm chooses

$$i_t \in \arg\max_i \mathrm{MMR}_i(S_t)$$

among candidates satisfying the remaining budget constraint.

This procedure continues until either (a) K_1 chunks have been selected, (b) no remaining chunk fits in the budget, or (c) all retrieved candidates have been exhausted. We deliberately describe the routine in prose rather than by an algorithm environment so that the source remains maximally portable across LaTeX installations.

Proposition 5.2 (Feasibility of greedy MMR)**.** *The selected set returned by the implemented MMR routine always satisfies both*

$$\sum_{R_i \in S} \ell_i \le B, \qquad |S| \le K_1.$$

Proof. The algorithm only appends a candidate if adding its word count does not exceed the current budget. Thus feasibility with respect to the budget is preserved inductively. The loop condition additionally stops when $|S| = K_1$, hence the cardinality constraint is also preserved. □

Remark 5.3. This proposition is elementary, but important. In numerical optimization language, the selector is a feasible descent-type heuristic on the discrete constraint set, even though it is not globally optimal.

6 Generation model and prompt-dependent approximation map

The generation model is

$$G_\theta = \texttt{google/flan-t5-base},$$

a sequence-to-sequence transformer. For a prompt string p, the model generates

$$\hat{y} = G_\theta(p),$$

with maximum output length

$$T_{\max} = \texttt{max_new_tokens} = 128.$$

The temperature is set to zero in the configuration, so decoding is effectively deterministic. The warning observed at runtime indicates that the passed temperature flag may be ignored by the underlying generation call; therefore deterministic generation should be understood operationally rather than axiomatically.

The repository defines two prompt maps.

6.1 Baseline prompt

If the packed context is c_{loc}, the prompt is

$$p_{\text{base}}(q, c_{\text{loc}}) = \text{instruction}(q, c_{\text{loc}}),$$

where the instruction asks the model to answer the question using only the provided context, concisely and in a grounded way.

Hence the baseline predictor is

$$\hat{y}_{\text{base}} = G_\theta\big(p_{\text{base}}(q, c_{\text{loc}})\big).$$

Since c_{loc} is just the top retrieved chunk, this system is a single-evidence approximation.

6.2 Context-aware prompt

If $g(D)$ denotes the global summary and c_{pack} the packed local evidence, then the prompt is

$$p_{\text{ctx}}(q, g, c_{\text{pack}}) = \text{instruction}(q, g, c_{\text{pack}}),$$

where the instruction explicitly separates global topic awareness from local factual grounding.

Therefore the context-aware predictor is

$$\hat{y}_{\text{ctx}} = G_\theta\big(p_{\text{ctx}}(q, g(D), c_{\text{pack}})\big).$$

Remark 6.1. The repository therefore approximates the latent problem not by changing model parameters θ, but by changing the conditioning map. In optimization language, it studies a *fixed-model, variable-context* regime.

7 Metric design

The repository evaluates each row by four metrics: ROUGE-L, support, term recall, and latency. These correspond closely to the latent utility components introduced earlier.

7.1 ROUGE-L

Let $y^\star$ be the reference answer and $\hat{y}$ the prediction. ROUGE-L is based on the longest common subsequence (LCS). If $\mathrm{LCS}(y^\star, \hat{y})$ denotes its length, then precision and recall are

$$P_{\mathrm{LCS}} = \frac{\mathrm{LCS}(y^\star, \hat{y})}{|\hat{y}|}, \qquad R_{\mathrm{LCS}} = \frac{\mathrm{LCS}(y^\star, \hat{y})}{|y^\star|},$$

and the F-measure is

$$\text{ROUGE-L}(y^\star, \hat{y}) = \frac{2P_{\mathrm{LCS}}R_{\mathrm{LCS}}}{P_{\mathrm{LCS}} + R_{\mathrm{LCS}}}.$$

The implementation uses the `rouge_score` library with stemming enabled.

Remark 7.1. ROUGE-L is a surface-form measure. It can reward lexical overlap even when the answer is incomplete or weakly grounded. Therefore it should never be interpreted in isolation.

7.2 Support score

The support score attempts to measure whether the generated answer is actually supported by the packed context. Let the prediction be split into sentences

$$\hat{y} = (\hat{s}_1, \ldots, \hat{s}_m)$$

and the packed context into sentences

$$c = (c_1, \ldots, c_n).$$

Each sentence is embedded by the same MiniLM sentence-transformer encoder, producing normalized vectors

$$\psi(\hat{s}_i), \quad \psi(c_j) \in \mathbb{R}^d.$$

For each predicted sentence, define its maximum contextual similarity

$$\kappa_i = \max_{1 \le j \le n} \langle \psi(\hat{s}_i), \psi(c_j) \rangle.$$

The support score is the mean

$$\mathrm{Supp}(\hat{y}; c) = \frac{1}{m} \sum_{i=1}^{m} \kappa_i.$$

Definition 7.2 (Support score)**.** *The support score is a sentence-level nearest-neighbor grounding functional. It is high when every generated sentence is close to at least one sentence in the packed context.*

Remark 7.3. This metric is asymmetrical. It measures whether the prediction is supported by the context, not whether the context fully covers the gold answer.

7.3 Term recall

Let $\mathrm{TopTerms}_{25}(D)$ denote the 25 most frequent non-stopword document terms extracted from the full document. Let $\mathrm{Terms}(\hat{y})$ be the set of content tokens in the prediction. The term recall metric is

$$\mathrm{TermRecall}(\hat{y}; D) = \frac{|\mathrm{TopTerms}_{25}(D) \cap \mathrm{Terms}(\hat{y})|}{|\mathrm{TopTerms}_{25}(D)|}.$$

This metric is intentionally simple. It measures whether the answer carries important document vocabulary.

7.4 Latency

The reported latency is

$$\mathrm{Latency}(\hat{y}) = t_{\mathrm{end}} - t_{\mathrm{start}},$$

where the time measurement is taken inside the generator around tokenization, forward generation, and decoding. It is therefore a *generation-stage latency*, not a full end-to-end pipeline latency. Retrieval, chunking, and summary construction are not included in this measurement.

Remark 7.4. This distinction matters. The context-aware system pays additional overhead before generation, but the stored latency in the current implementation records only the generation call. Consequently, the observed slowdown is already meaningful, and the true end-to-end slowdown would generally be no smaller.

8 Algorithmic summary

The full algorithm for one example may now be summarized mathematically.

8.1 Baseline system

Given (D, q):

(1) Construct chunks $\mathcal{C}(D)$.

(2) Compute retrieval scores $r_i = \langle \phi(q), \phi(C_i) \rangle$.

(3) Let $i^\star \in \arg\max_i r_i$ over the top-K_0 retrieved candidates.

(4) Set $c_{\mathrm{loc}} = C_{i^\star}$.

(5) Generate $\hat{y}_{\mathrm{base}} = G_\theta(p_{\mathrm{base}}(q, c_{\mathrm{loc}}))$.

8.2 Context-aware system

Given (D, q):

(1) Construct chunks $\mathcal{C}(D)$.

(2) Retrieve top-K_0 candidates using dense retrieval.

(3) Compute global summary $g(D)$ via sentence scoring.

(4) Apply greedy MMR under budget B and cardinality cap K_1 to obtain a selected set S.

(5) Pack local context by concatenation:

$$c_{\text{pack}} = \bigoplus_{R_i \in S} C_i.$$

(6) Generate

$$\hat{y}_{\text{ctx}} = G_\theta(p_{\text{ctx}}(q, g(D), c_{\text{pack}})).$$

9 Theoretical observations

Proposition 9.1 (Baseline is a degenerate one-step selector)**.** *The baseline system is a special case of the context-aware selection framework obtained by imposing $K_1 = 1$ and removing the global summary from the prompt.*

Proof. If $K_1 = 1$, the greedy MMR selector chooses a single chunk. When the selected set is empty, the redundancy term is zero, so the MMR score reduces to

$$\text{MMR}_i(\varnothing) = \lambda \langle e_i, e_q \rangle.$$

Since $\lambda > 0$, maximizing MMR is equivalent to maximizing retrieval relevance. Hence the selected chunk is exactly the top retrieved chunk. Removing the global summary from the prompt yields the baseline system. □

Proposition 9.2 (Potential failure mode of context expansion)**.** *Assume the generator G_θ is imperfect and not strictly monotone with respect to conditioning information. Then there exist documents and queries for which enlarging the prompt context decreases grounding quality.*

Proof. Consider two prompts p_1 and p_2 with p_2 containing all information in p_1 plus additional semantically broad but weakly discriminative context. Since G_θ is a nonlinear conditional generator, there is no guarantee that

$$G_\theta(p_2)$$

will preserve the local evidence emphasis present in

$$G_\theta(p_1).$$

In particular, if the added context shifts the model toward a generic summary mode, then the answer may become less extractive and less directly supported by the packed evidence. Therefore support can decrease even as lexical overlap with the reference changes. □

Remark 9.3. This proposition is simple but central for interpreting the measured experiment. More context is not mathematically synonymous with better conditioning.

10 Numerical data description

The dataset is QMSum, a meeting summarization and query-focused summarization corpus. In the executed protocol, examples are built from the canonical JSONL serialization of the corpus rather than from the incompatible Hugging Face split interface originally referenced by the repository. Each record contains:

- a meeting transcript,
- a list of general queries,

- a list of specific queries,
- a gold answer for each query.

The loader flattens each meeting into multiple supervised examples of the form

$$(D, q, y^\star).$$

For the present experiment, the run command used

$$\texttt{split=train}, \qquad \texttt{max-samples} = 5.$$

Thus the numerical results reported below correspond to the first five extracted supervised examples.

10.1 Observed qualitative behavior

The examples inspected from the output JSONL files show the following pattern:

- the baseline often produces short fragmentary outputs strongly tied to one local chunk;
- the context-aware system often produces syntactically smoother but semantically generic outputs;
- the added global summary and packed local context do not, on this sample, translate into richer factual answers.

This qualitative pattern already suggests a tension between *surface plausibility* and *factual specificity*.

11 Numerical results

The computed metrics are reproduced below.

System	ROUGE-L	Support	Term Recall	Latency (s)
Baseline	0.060181	0.747300	0.056000	0.473008
Context-aware	0.106888	0.589833	0.032000	2.674552

11.1 Primary quantitative observations

ROUGE-L. The context-aware system improves ROUGE-L from

$$0.060181 \longrightarrow 0.106888.$$

In relative terms,

$$\frac{0.106888 - 0.060181}{0.060181} \approx 0.776,$$

which is about a 77.6% increase. However, the absolute values remain low, indicating that both systems are still far from high-fidelity answer reconstruction.

Support. Support decreases from

$$0.747300 \longrightarrow 0.589833.$$

The absolute drop is

$$0.747300 - 0.589833 = 0.157467,$$

which indicates that the context-aware answers are less tightly aligned to the packed evidence than the baseline answers.

Term recall. Term recall decreases from

$$0.056000 \longrightarrow 0.032000,$$

so the context-aware outputs recover fewer of the document's top terms. This strongly suggests loss of key content vocabulary.

Latency. Generation latency increases from

$$0.473008 \text{ s} \longrightarrow 2.674552 \text{ s}.$$

The measured slowdown factor is

$$\frac{2.674552}{0.473008} \approx 5.654.$$

Thus the context-aware system is about $5.65\times$ slower in the recorded generation stage alone.

Remark 11.1. This is exactly the kind of outcome that we expect in multi-objective systems. Once several competing objectives are present, there is no reason to expect monotone improvement in all coordinates of the objective vector.

Let us express this more formally. If we define the empirical metric vector

$$M(\hat{y}) = (\text{ROUGE-L}, \text{Supp}, \text{TermRecall}, -\text{Latency}),$$

then the context-aware result is not Pareto-superior to the baseline result. It improves the first coordinate and worsens the remaining three. Therefore the experiment reveals a nontrivial Pareto trade-off rather than dominance.

11.2 Why this matters

A superficial reading would say that context-aware retrieval "helps" because ROUGE-L improved. A careful scientific reading says the opposite: the added context altered the behavior of the generator, but the resulting outputs became less grounded and less terminologically informative while becoming much slower. Thus the principal bottleneck is likely not raw retrieval breadth alone, but the interaction between prompt design, model capacity, and the structure of the packed context.

The repository may be interpreted as solving a nested approximation problem:

$$\begin{aligned}
\text{document} &\xrightarrow{\text{chunking}} \text{discrete candidates} \\
&\xrightarrow{\text{dense retrieval}} \text{continuous relevance scores} \\
&\xrightarrow{\text{greedy MMR}} \text{feasible sparse subset} \\
&\xrightarrow{\text{prompt map}} \text{conditioned seq2seq generation} \\
&\xrightarrow{\text{metrics}} \text{multi-objective evaluation.}
\end{aligned}$$

Each stage introduces approximation error.

(1) Chunking discretizes the continuous discourse structure of the document.

(2) Dense retrieval replaces symbolic relevance by embedding similarity.

(3) Greedy MMR approximates a difficult combinatorial packing problem.

(4) Prompting converts structured evidence into a flat textual conditioning string.

(5) Generation maps this text into an answer under a fixed pretrained model.

The present numerical results imply that improving one intermediate stage, namely context packaging, is not sufficient to guarantee net improvement of the end-to-end objective. In inverse-problem language, the forward map from selected context to generated answer is itself imperfect and unstable with respect to perturbations in the prompt.

12 Limitations and next steps

The present experiment is intentionally small, and therefore its conclusions are diagnostic rather than definitive. The principal limitations are:

- only 5 examples were evaluated;
- the generator is a relatively small FLAN-T5 model;
- latency is generation-stage only, not full pipeline latency;
- the global summary is heuristic rather than learned;
- the greedy MMR selector optimizes a surrogate objective, not the true end-to-end utility.

These limitations naturally lead to next steps.

(a) **Larger sample regime.** Estimate whether the observed trade-off persists under more examples. This turns the current point estimate into a statistical study.

(b) **Prompt optimization.** Replace the current prompt by a family p_η with tunable parameters and optimize

$$\max_{\eta} \mathbb{E}\, \mathcal{J}\big(G_\theta(p_\eta), D, q\big).$$

(c) **Generator scaling.** Study whether a stronger generator better exploits the richer context.

(d) **End-to-end selection.** Replace the heuristic MMR rule by a learned or differentiable surrogate coupled more directly to downstream answer quality.

13 Conclusion

This report has formulated the repository `wordcopilot-context-lab` as a constrained multi-objective approximation problem. The baseline system is a single-chunk local-context method. The context-aware system enriches the conditioning signal through a global summary and a greedy MMR-selected packed context. Both systems use the same dense retriever and the same FLAN-T5 generator, so the experiment isolates the effect of context construction rather than model retraining.

The measured numerical result is clear: on the executed five-example QMSum sample, the context-aware system improves ROUGE-L but worsens support, term recall, and latency, with an approximately 5.65× increase in measured generation time. The context-aware point is not a dominant point in objective space. It lies on a trade-off frontier.